

GW2 Game Server

C++ 멀티플레이 게임 서버 포트폴리오

C++ IOCP

Protobuf

A* 경로탐색

Unity Client

Server-Auth

RTS / MOBA | 서버 권위 구조 | JobQueue 스레드 안전 설계

프로젝트 개요

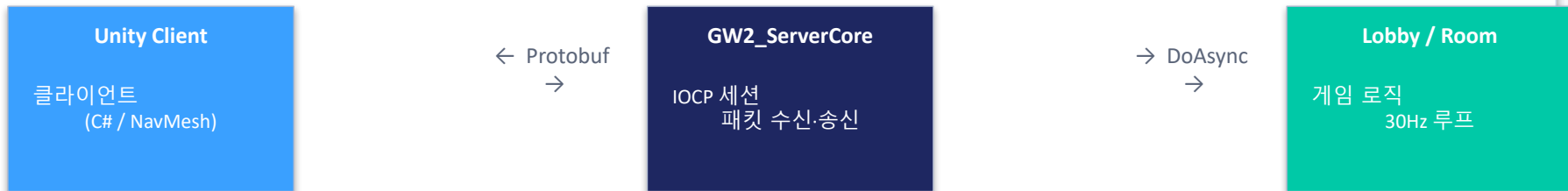
프로젝트 소개

- RTS/MOBA 장르의 C++ 멀티플레이 게임 서버
- 서버 권위(Server-Authoritative) 구조
- 모든 게임 로직은 서버에서 계산·검증
- 클라이언트는 Unity로 결과 표현만 담당
- Protobuf 직렬화 + IOCP 비동기 네트워크

기술 스택

서버	C++17 · IOCP · Protobuf
AI	A* · WalkableGrid · 상태머신
네트워크	JobQueue · DoAsync 패턴
클라이언트	Unity · NavMesh · C#

전체 구성



서버 아키텍처

GW2_ServerCore

IOCP · GameSession · 패킷 송수신

Lobby (GLobby 싱글톤)

NavMesh 로드 · Room 관리 · Stats 로드

Room (JobQueue 상속)

게임 루프 · Handler 함수 · Broadcast

Object 계층

Player · Minion · Turret · Nexus · Baron

핵심 설계 원칙

스레드 안전

Room JobQueue로 직렬 실행
모든 Room 로직 단일 스레드

DoAsync 패턴

크로스 스레드 호출 시
반드시 DoAsync 사용

30 Hz 게임 루프

Lobby가 33ms Tick으로
모든 Room UpdateRoom 호출

서버 권위

이동·전투·버프 모두 서버
클라이언트는 표현만 담당

스레드 모델 & JobQueue

IOCP Worker Thread

- 네트워크 I/O 처리
- 패킷 수신 → 디스패치
- GameSession 관리
- 비동기 Send/Recv

Lobby Main Thread

- RunRooms() — 30Hz Tick
- LobbyUpdate(deltaTime)
- room→DoAsync 디스패치
- Room 생성·삭제 관리

Room JobQueue

- 직렬 실행 보장
- HandleSkill / HandleMove
- UpdateRoom(dt)
- Broadcast 처리

DoAsync 패턴 — 크로스 스레드 안전 호출

```
// ✅ 안전: Minion 스레드에서 Room 함수 호출
room->DoAsync(&Room::HandleMinionAttack, self, currentTarget);
// ❌ 위험: 직접 호출 — 절대 금지 room->HandleMinionAttack(self, currentTarget);
```

이동 시스템 — 서버 권위 + 클라이언트 예측

서버 이동 처리 흐름

① C_MOVE 수신

출발·목적지 좌표 전달

② ValidateMove()

walkable 여부 검증

③ A* FindPath()

WalkableGrid 경로탐색

④ SmoothPath()

불필요 꺾임 제거

⑤ S_MOVE 브로드캐스트

100ms(10Hz) 주기 위치 동기화

⑥ S_MOVE_END

경로 완주 시 최종 위치 확정

클라이언트 예측 (MyPlayer)

- C_MOVE 전송과 동시에 NavMesh로 경로 예측
- NavMeshAgent.updatePosition = false
- transform.position 수동 제어
- S_MOVE_END 수신 시 3m 초과 차이만 보정

텔레포트 처리

Mobility 카드 조건 (서버):

buffType==0 && damage==0 && heal==0

→ 즉시 SetPosVector() + S_MOVE_END

클라이언트: diff > 3.0m → agent.Warp()

NavMesh 위에 올바르게 착지 보장

미니언 AI — 상태머신



_pathPending 가드

DoAsync 중복 요청 방지
LINE_TRACE: RAII 자동 해제
CHASE: 수동 ClearPathPending()

레인 필터 (laneId)

allowedLaneId=0: 전체 허용
allowedLaneId=1: 상단 레인
allowedLaneId=3: 하단 레인

타겟 우선순위

Turret (3) > Minion (2)
> Player/Nexus (1)
아군 = -1 (제외)

leash 거리

detectionRange × 1.5 이탈
→ LINE_TRACE 복귀
1.5초 재탐지 유예

카드/스킬 시스템

카드 분류 (target_type)

Self (0)

즉시 자신에게 적용
버프·회복 카드

ClickPoint (1)

클릭 지점으로 이동
텔레포트·AOE 카드

ClickTarget (2)

좌클릭 적 대상
단일 공격·스턴 카드

HandleSkill() 처리 흐름

C_SKILL 수신

target_id == 0 ?

카드 소유·사용 처리

Heal 처리 → S_HP_CHANGE

버프 처리 → S_BUFF_APPLIED

S_SKILL 브로드캐스트

Mobility? → 즉시 이동

———— or ————

타겟 검증 (팀·사망·거리)

데미지 계산 + 방어 감소

S_HP_CHANGE + S_DIE

버프 시스템

BuffAttack (1)

서버: `_attackMult = value`
(예: 1.5 = 150%)

`Player.cpp:71`
`_attackBuffTimer` 타이머 관리

BuffDefense (2)

서버: `_defenseReduct = value`
(예: 0.3 = 30% 피해 감소)

`Room.cpp:488`
`damage × (1 - _defenseReduct)`

BuffSpeed (3)

서버: `_speedMult = value`
(예: 2.0 = 이동속도 2배)

`Player.cpp:119`
`remaindist × _speedMult`

BuffAttackSpeed (4)

서버: `_attackSpeedMult = value`
(예: 0.5 = 쿨타임 절반)

`Room.cpp:380`
`interval / _attackSpeedMult`

서버 → 클라이언트 버프 흐름



※BuffDefense value: 0.3 = 30% 감소 비율. UI 표시 시 $\times 100$ 변환 필요 (`_shield = value \times 100f`)

게임 루프 & 세션 생명주기

세션 생명주기



UpdateRoom(deltaTime) — 30Hz

- ① 각 Object::UpdateController(dt)
Player: 경로이동 + 버프 타이머 감소
Minion: 상태머신 (IDLE→ATTACK)
- ② S_MOVE 브로드캐스트 (100ms 주기)
- ③ 미니언 웨이브 스폰 (30초 간격)
- ④ 리스폰 타이머 감소
- ⑤ 골드 자동 지급 (1초, +20 Gold)

타이밍 요약

게임 틱	33ms (30Hz)
S_MOVE 브로드캐스트	100ms (10Hz)
미니언 웨이브	30초 간격
골드 자동 지급	1초 / +20 Gold
플레이어 리스폰	사망 후 5초
미니언 타겟 탐색	0.1초 간격

구현을 통해 배운 것들

서버 권위 아키텍처

모든 게임 로직을 서버에서 처리하는 구조 설계.
클라이언트 예측과 서버 보정을 동시에 구현.

JobQueue 기반 스레드 안전 설계

DoAsync 패턴으로 크로스 스레드 호출을 직렬화.
Race condition 없이 멀티스레드 환경 운용.

A* 경로탐색 & 미니언 AI

WalkableGrid + lanelId 필터로 레인 기반 탐색 구현.
상태머신으로 IDLE→CHASE→ATTACK 전환 관리.

Protobuf 기반 패킷 설계

클라이언트-서버 간 타입 안전한 메시지 직렬화.
패킷 방향(C/S_) 규칙으로 흐름 명확화.